

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра «Програмна інженерія»
Звіт з лабораторної роботи №2
З дисципліни "Генеративні нейронні мережі та їх застосування"
Базовий Denoising diffusion probabilistic model

Виконала:
студентка гр. ПЗМ-25-2
Тимощенко Владислава Русланівна

Перевірила: доцент кафедри ПІ
Шевченко Олена Леонідівна

Харків 2026

1 Мета: Навчитися реалізовувати та використовувати базову DDPM на основі бібліотеки PyTorch

2 Хід роботи

За основу лабораторної роботи пропонується взяти статтю Vaibhav Singh An In-Depth Guide to Denoising Diffusion Probabilistic Models DDPM – Theory to Implementation, 2023 р. (URL: <https://learnopencv.com/denoising-diffusion-probabilistic-models/>).

В якості прикладу вихідного коду до статті використаємо наступний репозиторій: <https://github.com/gctian/DDPMs-from-scratch>

На оцінку «задовільно»

В якості набору даних для полегшення навчання та тестування візьмемо MNIST

2.1 Впевнімося, що image size скрізь налаштоване на 32×32

```
class SimpleDiffusion:
    def __init__(
        self,
        num_diffusion_timesteps=1000,
        img_shape=(3, 32, 32),
        device="cpu",
    ):
        self.num_diffusion_timesteps = num_diffusion_timesteps
        self.img_shape = img_shape
        self.device = device
```

```
checkpoint_dir = version_0

@dataclass
class TrainingConfig:
    TIMESTEPS = 1000 # Define number of diffusion timesteps
    IMG_SHAPE = (1, 32, 32) if BaseConfig.DATASET == "MNIST" else (3, 32, 32)
    NUM_EPOCHS = 30
    BATCH_SIZE = 128
    LR = 2e-4
    NUM_WORKERS = 2
```

```
# Algorithm 2: Sampling

@torch.inference_mode()
def reverse_diffusion(model, sd, timesteps=1000, img_shape=(3, 32, 32),
                     num_images=5, nrow=8, device="cpu", **kwargs):

    x = torch.randn((num_images, *img_shape), device=device)
    model.eval()
```

Load Dataset & Build DataLoader

```
def get_dataset(dataset_name='MNIST'):
    transforms = TF.Compose(
        [
            TF.ToTensor(),
            TF.Resize((32, 32),
                      interpolation=TF.InterpolationMode.BICUBIC,
                      antialias=True),
            # TF.RandomHorizontalFlip(),
            TF.Lambda(lambda t: (t * 2) - 1) # Scale between [-1,
```

2.2 Замість 1000 timesteps зробимо 100 на training та стільки ж на sampling

```
# Algorithm 2: Sampling

@torch.inference_mode()
def reverse_diffusion(model, sd, timesteps=100, img_shape=(3, 32, 32),
                     num_images=5, nrow=8, device="cpu", **kwargs):

    x = torch.randn((num_images, *img_shape), device=device)
    model.eval()
```

```
reverse_diffusion(
    model,
    sd,
    num_images=64,
    generate_video=generate_video,
    save_path=save_path,
    timesteps=100,
    img_shape=TrainingConfig.IMG_SHAPE,
    device=BaseConfig.DEVICE,
    nrow=8,
)
print(save_path)
```

Diffusion Process

```
class SimpleDiffusion:
    def __init__(
        self,
        num_diffusion_timesteps=100,
        img_shape=(3, 32, 32),
        device="cpu",
    ):
        self.num_diffusion_timesteps = num_diffusion_timesteps
        self.img_shape = img_shape
        self.device = device
```

```

class SinusoidalPositionEmbeddings(nn.Module):
    def __init__(self, total_time_steps=100, time_emb_dims=64):
        super().__init__()

        half_dim = time_emb_dims // 2

        emb = math.log(10000) / (half_dim - 1)
        emb = torch.exp(torch.arange(half_dim, dtype=torch.float) * emb)

        emb = emb.to(device)

```

```

x0s, _ = next(loader)

noisy_images = []
specific_timesteps = [0, 1, 5, 10, 15, 20, 25, 30, 40, 60, 80, 99]

for timestep in specific_timesteps:
    timestep = torch.as_tensor(timestep, dtype=torch.long)

    xts = forward_diffusion(sd, x0s, timestep)

```

2.3 Залишимо 32 базових канали

```

pil_image.save(kwargs['save_path'], format=save_path[-3:])
display(pil_image)
return None

@dataclass
class ModelConfig:
    BASE_CH = 32 # 64, 128, 256, 512
    BASE_CH_MULT = (1, 2, 4, 8) # 32, 16, 8, 4
    APPLY_ATTENTION = (False, False, True, False)
    DROPOUT_RATE = 0.1
    TIME_EMB_MULT = 2 # 128

model = UNet(

```

```

class UNet(nn.Module):
    def __init__(
        self,
        input_channels=3,
        output_channels=3,
        num_res_blocks=2,
        base_channels=32,
        base_channels_multiples=(1, 2, 4, 8),
        apply_attention=(False, False, True, False),
        dropout_rate=0.1,
        time_multiple=4,
    ):
        super().__init__()

```

2.4 Приберемо attention з UNet

```
class UNet(nn.Module):
    def __init__(
        self,
        input_channels=3,
        output_channels=3,
        num_res_blocks=2,
        base_channels=32,
        base_channels_multiples=(1, 2, 4, 8),
        apply_attention=(False, False, False),
        dropout_rate=0.1,
        time_multiple=4,
    ):
        super().__init__()
```

2.5 Кількість рівнів UNet замінимо з 4 на 3

```
@dataclass
class ModelConfig:
    BASE_CH = 32 # 64, 128, 256, 512
    BASE_CH_MULT = [1, 2, 4] # 32, 16, 8, 4
    APPLY_ATTENTION = (False, False, False)
    DROPOUT_RATE = 0.1
    TIME_EMB_MULT = 2 # 128
```

2.6 Замінимо 2 residual-блоки на 1 на рівень

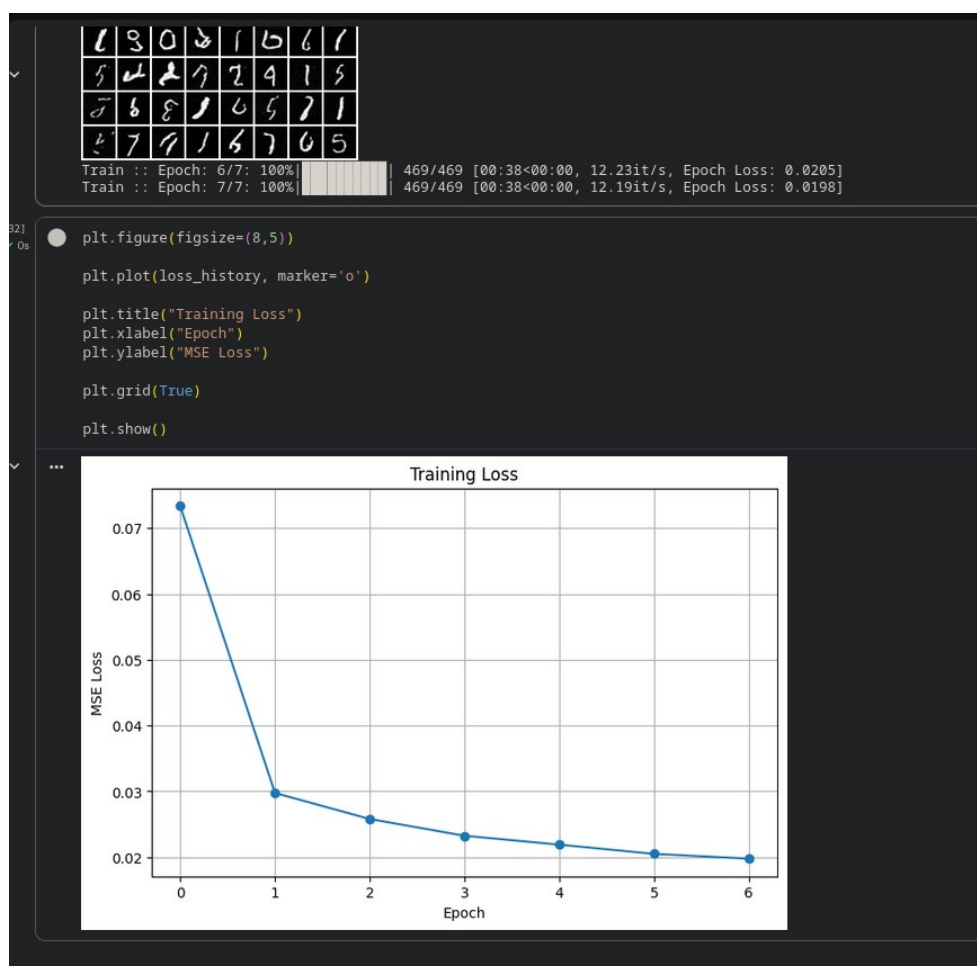
```
input_channels=3,
output_channels=3,
num_res_blocks=1,
base_channels=32,
base_channels_multiples=(1, 2, 4, 8),
apply_attention=(False, False, False),
dropout_rate=0.1,
time_multiple=4,
):
    super().__init__()
```

2.7 Замінімо 30 епох навчання на 5-10

```
@dataclass
class TrainingConfig:
    TIMESTEPS = 1000 # Define number of diffusion timesteps
    IMG_SHAPE = (1, 32, 32) if BaseConfig.DATASET == "MNIST" else (3, 32, 32)
    NUM_EPOCHS = 7
    BATCH_SIZE = 128
    LR = 2e-4
    NUM_WORKERS = 2
```

2.8 Додамо графік loss

Після тренування моделі створимо нову клітинку, яка буде генерувати графік з отриманих даних:



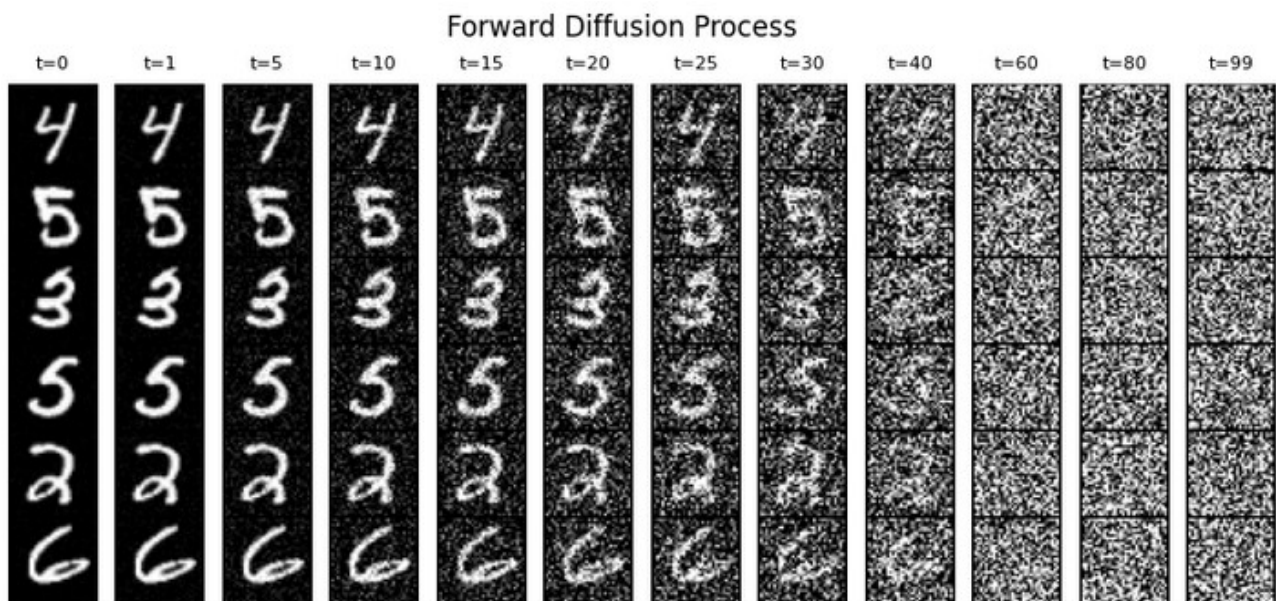
Під час навчання DDPM-моделі на наборі даних MNIST спостерігається стабільне зменшення значення функції втрат від епохи до епохи. На першій епосі середнє значення loss становило 0.0734, а на сьомій зменшилося до 0.0198.

Найбільше покращення відбулося на початкових етапах навчання: між першою та другою епохами loss зменшився більш ніж удвічі (з 0.0734 до 0.0298). Надалі зниження продовжувалося більш плавно, що свідчить про поступове наближення моделі до оптимального розв'язку.

Монотонне зменшення loss без різких стрибків або зростання вказує на стабільний процес навчання та успішне засвоєння моделлю структури даних MNIST. Отримані результати свідчать про те, що архітектура UNet та вибрані параметри дифузійного процесу дозволяють ефективно навчати модель для генерації рукописних цифр.

Таким чином, графік функції втрат підтверджує коректність процесу навчання та демонструє поступове покращення якості прогнозування шуму зі збільшенням кількості епох.

2.9 Додамо візуалізацію forward diffusion



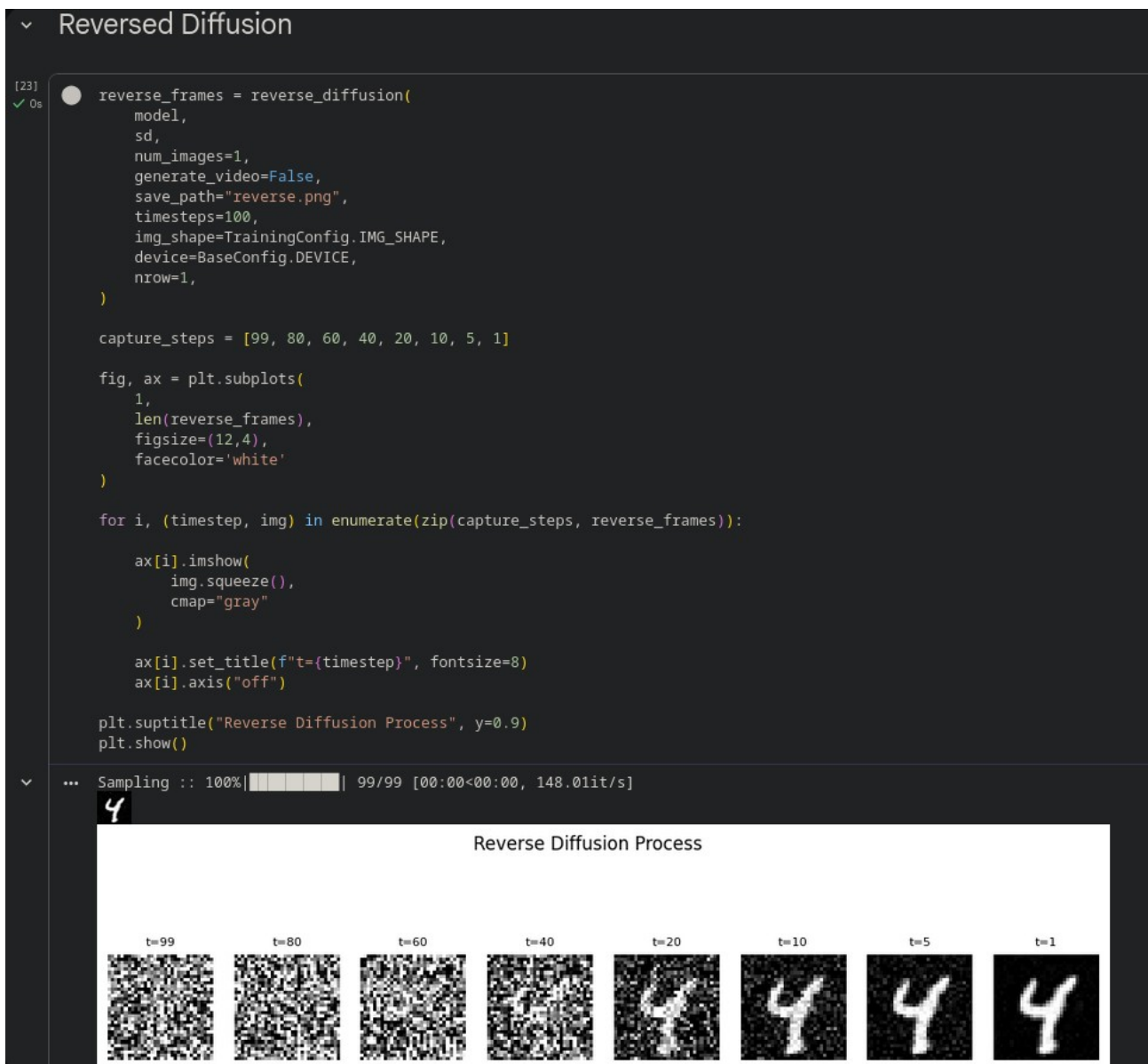
На рисунку представлено процес прямої дифузії (Forward Diffusion) для набору даних MNIST. На кожному часовому кроці до зображення поступово додається гаусів шум відповідно до розкладу шуму (noise schedule).

На початкових кроках ($t = 0-30$) структура рукописної цифри майже повністю зберігається, а шум є незначним. Із подальшим збільшенням значення t рівень шуму зростає, що призводить до поступової втрати інформації про

початкове зображення. На пізніх етапах процесу ($t = 70\text{--}99$) контури цифри практично зникають, а зображення наближається до випадкового гаусового шуму.

Отриманий результат підтверджує коректну роботу прямого дифузійного процесу та демонструє поступове руйнування структури зображення, що є основною ідеєю дифузійних генеративних моделей.

2.10 Додамо візуалізацію reverse diffusion



На рисунку представлено процес зворотної дифузії (Reverse Diffusion) у моделі DDPM. На відміну від прямої дифузії, де до зображення поступово

додається шум, зворотна дифузія починається з випадкового гаусового шуму та послідовно відновлює структуру зображення.

На кожному часовому кроці нейронна мережа UNet прогнозує шумову складову, яка була додана на відповідному етапі прямої дифузії. Після цього виконується її усунення, що дозволяє поступово переходити від випадкового шуму до осмисленого зображення.

На початкових кроках ($t = 99, 80$) зображення має вигляд майже чистого шуму, без видимих ознак цифри. У середній частині процесу ($t = 60, 40, 20$) починають формуватися контури майбутнього об'єкта та з'являються характерні елементи рукописної цифри. На завершальних етапах ($t = 10, 5, 1$) модель успішно відновлює чітке зображення цифри, яке відповідає розподілу даних навчального набору MNIST.

Отримані результати демонструють коректну роботу механізму зворотної дифузії та підтверджують здатність моделі генерувати нові зображення шляхом поступового видалення шуму з випадкового початкового стану.

2.11 Додамо grid зі згенерованих 16 зображень

```
[42] generate_video = True

ext = ".mp4" if generate_video else ".png"
filename = f"{datetime.now().strftime('%Y%m%d-%H%M%S')}{ext}"

save_path = os.path.join(log_dir, filename)

reverse_diffusion(
    model,
    sd,
    num_images=16,
    generate_video=generate_video,
    save_path=save_path,
    timesteps=100,
    img_shape=TrainingConfig.IMG_SHAPE,
    device=BaseConfig.DEVICE,
    nrow=4,
)
print(save_path)
```

... Sampling :: 100% | 99/99 [00:00<00:00, 137.23it/s]



inference_results/20260603-172137.mp4

3. Висновки: в ході роботи було реалізовано та навчено дифузійну модель DDPM на наборі даних MNIST із використанням спрощеної архітектури UNet. Аналіз графіка функції втрат показав стабільне зменшення loss протягом навчання, що свідчить про успішне засвоєння моделлю структури даних. Візуалізації процесів forward diffusion та reverse diffusion підтвердили коректну роботу механізму додавання та видалення шуму, а також здатність моделі генерувати нові зображення рукописних цифр.